Robotic Manipulation

Perception, Planning, and Control

Russ Tedrake

© Russ Tedrake, 2020-2026
Last modified 2026-10-11.
How to cite these notes, use annotations, and give feedback.

Note: These are working notes used for a course being taught at MIT. They will be updated throughout the Fall 2026 semester.

Previous Chapter Table of contents Next Chapter

Chapter 2
Let's get you a robot

In this chapter we're going to outfit your mech. I want to make sure you understand the robot hardware that we're using in these notes, and how it compares to the other hardware available today. You should also come away with an understanding of how we simulate the robot and what commands you can send to the robot interface.

2.1 Robot description files

In the majority of the chapters, we'll make repeated use of just one or two robot configurations. One of the great things about modern robotics is that many of the tools we will develop over the course of these notes are quite general, and can be transferred from one robot to another easily. I could imagine a future version of these notes where you really do get to build out your robot in this chapter, and use your customized robot for the remaining chapters!

The ability to easily simulate/control a variety of robots is made possible in part by the proliferation of common file formats for describing our robots. Unfortunately, the field has not converged on a single preferred format (yet), and each of them have their quirks. Drake currently loads Universal Robot Description Format (URDF), Simulation Description Format (SDF), and has limited support for the MuJoCo format (MJCF). The Drake developers have been trying to upstream improvements to SDF rather than start yet another format, but we do have a very simple YAML specification called Drake Model Directives which makes it very quick and easy to load multiple robots/objects from these different file formats into one simulation; you saw an example of this in the introduction chapter notebook.

2.2 Arms

There appear to be a lot of robotics arms available on the market. So how does one choose? Cost, reliability, usability, payload, range of motion, ...; there are many important considerations. And the choices we might make for a research lab might be very different than the choices we might make as a startup.

Example 2.1 (Robot arms)

I've put together a simple example to let you explore some of the various robot arms that are popular today. Let me know if your favorite arm isn't on the list yet!

Open in Colab

There is one particular requirement which, if we want our robot to satisfy, quickly winnows the field to only a few viable platforms: that requirement is joint-torque sensing and control. Out of the torque-controlled robots on the market, I've used the Kuka LBR iiwa robot most often throughout these notes (I will try to use the lower case "iiwa" to be consistent with the manufacturer, but it looks wrong to me every time!).

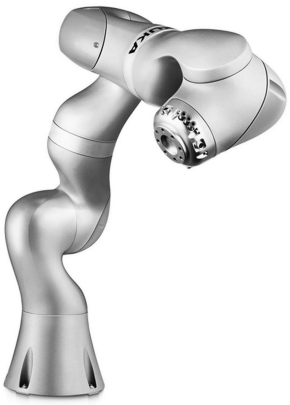


Figure 2.1 - Kuka LBR iiwa robot. This one has a 7kg payload.

It's not absolutely clear that the joint-torque sensing and control feature is required, even for very advanced applications, but as a researcher who cares a great deal about the contact interactions between my robots and the world, I prefer to have the capability and explore whether I need it rather than wonder what life might have been like. To better understand why, let us start by understanding the difference between most robots, which are position-controlled, and the handful of robots that have accepted the additional cost and complexity to provide torque sensing and control.

2.2.1 Position-controlled robots



Figure 2.2 - Two popular position controlled manipulators. (Left) The UR10 from Universal Robotics. (Right) The ABB Yumi.

Most robot arms today are "position controlled" -- given a desired joint position (or joint trajectory), the robot executes it with relatively high precision. Basically all arms can be position controlled -- if the robot offers a torque control interface (with sufficiently high bandwidth) then we can certainly regulate positions, too. In practice, calling a robot "position controlled" is a polite way of saying that it does not offer torque control. Do you know why position control and not torque control is the norm?

Lightweight arms like the examples above are actuated with electric motors. For a reasonably high-quality electric motor (with windings designed to minimize torque ripple, etc), we expect the torque that the motor outputs to be directly proportional to the current that we apply: $$\tau_{motor} = k_t i,$$ where τ_{motor} is the motor

torque, i is the applied current, and k_t is the "motor torque constant". (Similarly, applied voltage has a simple (affine) relationship with the motor's steady-state velocity). If we can control the current, then why can we not control the torque?

The short answer is that to achieve reasonable cost and weight, we typically choose small electric motors with large gear reductions, and gear reductions come with a number of dynamic effects that are very difficult to model -- including backlash, vibration, and friction. So the simple relationship between current and torque breaks down. Conventional wisdom is that for large gear ratios (say $\gg 10$), the unmodeled terms are significant enough that they cannot be ignored, and torque is no longer simply related to current.

Position Control.

How can we overcome this challenge of not having a good model of the transmission dynamics? Regulating the current or speed *of the motor* only requires sensors on the motor side of the transmission. To accurately regulate the joint, we typically need to add more sensors on the output side of the transmission. Importantly, although the torques due to the transmission are not known precisely, they are also not arbitrary -- for instance they will never add energy into the system. Most importantly, we can be confident that there is a *monotonically increasing* relationship between the current that we put into the motor and the torque at the joint, and ultimately the acceleration of the joint. Note that I chose the term monotonic carefully, meaning "non-decreasing" but *not* implying "strictly increasing", because, for instance, when a joint is starting from rest, static friction will resist small torques without having any acceleration at the output.

The most common sensor to add to the joint is a position sensor -- typically an encoder or potentiometer -- these are inexpensive, accurate, and robust. In practice, we think of these as providing (after some signal filtering/conditioning) accurate measurements of the joint position and joint velocity -- joint accelerations can also be obtained via differentiating twice but are generally considered more noisy and less suitable for use in tight feedback loops. Position sensors are sufficient for accurately tracking desired position trajectories of the arm. For each joint, if we denote the joint position as q and we are given a desired trajectory $q^d(t)$, then I can track this using proportional-integral-derivative (PID) control: $$\tau = k_p (q^d - q) + k_d (\dot{q}^d - \dot{q}) + k_i \int (q^d - q) dt,$$ with k_p, k_d, and k_i being the position, velocity, and integral gains. PID control has a rich theory, and a trove of knowledge about how to choose the gains, which I will not reproduce here. I will note, however, that when we simulate position-controlled robots we often need to use different gains for the physical robot and for our simulations. This is due to the transmission dynamics, but also the fact that PID controllers in hardware typically output voltage commands (via pulse-width modulation) instead of current commands. Closing this modeling gap has traditionally not been a priority for robot simulation -- there are enough other details to get right which dominate the "sim-to-real" gap -- but I suspect that as the field matures the mainstream robotics simulators will eventually capture this, too.

Some of you are thinking, "I can train a neural network to model *anything*, I'm not afraid of difficult-to-model transmissions!" I do think there is reason to be optimistic about this approach; there are a number of initial demonstrations in this direction (e.g. Hwangbo19). This is not quite as useful as if we can have a first-principles model that can generalize to new actuators from a few parameters in a description file, but could be very productive.

An aside: link dynamics with a transmission.

One thing that might be surprising is that, despite the fact that the joint dynamics of a manipulator are highly coupled and state dependent, the PID gains are often chosen independently for each joint, and are constant (not gain-scheduled). Wouldn't you expect for the motor commands required for e.g. a robot arm at full extension holding a milk jug might be very different than the motor commands required when it is unloaded in a vertical hanging position? Surprisingly, the required gains/commands might not be as different as one would think.

Electric motors are most efficient at high speeds (often > 100 or 1,000 rotations per minute). We probably don't actually want our robots to move that fast even if they could! So nearly all electric robots have fairly substantial gear reductions, often on the order of 100:1; the transmission output turns one revolution for every 100 rotations of the motor, and the output torque is 100 times greater than the motor torque. For a gear ratio, n, actuating a joint q, we have $$q_{motor} = n q,\quad \dot{q}_{motor} = n \dot{q}, \quad \ddot{q}_{motor} = n \ddot{q}, \qquad \tau_{motor} = \frac{1}{n} \tau.$$ Interestingly, this has a fairly profound impact on the resulting dynamics (given by $f = ma$), even for a single joint. Writing the relationship between joint torque and joint acceleration (no motors yet), we can write $ma = \sum f$ in the rotational coordinates as $$I_{arm} \ddot{q} = \tau_{gravity} + \tau,$$ where I_{arm} is the moment of inertia. For example, for a simple pendulum, we might have $$ml^2 \ddot{q} = - mgl\sin{q} + \tau.$$ But the applied joint torque τ actually comes from the motor -- if we write this equation in terms of motor coordinates we get: $$\frac{I_{arm}}{n} \ddot{q}_{motor} = \tau_{gravity} + n\tau_{motor}.$$ If we divide through by n, and take into account the fact that the motor itself has inertia (e.g. from the large spinning magnets) that is not affected by the gear ratio, then we obtain: $$\left(I_{motor} + \frac{I_{arm}}{n^2}\right) \ddot{q}_{motor} = \frac{\tau_{gravity}}{n} + \tau_{motor}.$$

It's interesting to note that, even though the mass of the motors might make up only a small fraction of the total mass of the robot, for highly geared robots they can play a significant role in the dynamics of the joints. We use the term *reflected inertia* to denote the inertial load that is felt on the opposite side of a transmission, due to the scaling effect of the transmission. The "reflected inertia" of the arm at the motor is cut by the square of the gear ratio; or the "reflected inertia" of the motor at the arm is multiplied by the square of the gear ratio. This has interesting consequences -- as we move to the multi-link case, we will see that I_{arm} is a state-dependent function that captures the inertia of the actuated link and also the inertial coupling of the other joints in the manipulator. I_{motor}, on the other hand, is constant and only affects the local joint. For large gear ratios, the I_{motor} terms dominate the other terms, which has two important effects: 1) it effectively diagonalizes the manipulator equations (the inertial coupling terms are relatively small), and 2) the dynamics are relatively constant throughout the workspace (the state-dependent terms are relatively small). These effects make it relatively easy to tune constant feedback gains for each joint individually that perform well in all configurations.

2.2.2 Torque-controlled robots

Although not as common, there are a number of robots that do support direct control of the joint torques. There are a handful of ways that this capability can be realized.

It *is* possible to actuate a robot using electric motors that require only a small gear reduction (e.g. $\le$ 10:1) where the frictional forces are negligible. In the past, these

"direct-drive robots"Asada87 had enormous motors and limited payloads. More recently, robots like the Barrett WAM arm used cable drives to keep the arm light by having large motors in the base. And just in the last few years, we've seen progress in high-torque outrunner and frameless motors bringing in a new generation of low-cost, "quasi-direct-drive" robots: e.g. MIT Cheetah Wensing17, Berkeley Blue, and Halodi Eve.

Hydraulic actuators provide another solution for generating large torques without large transmissions. Sarcos had a series of torque-controlled arms (and humanoids), and many of the most famous robots from Boston Dynamics are based on hydraulics (though there is an increasing trend towards electric motors). These robots typically have a single central pump and each actuator has a (lightweight) valve that can shunt fluid through the actuator or through a bypass; the differential pressure across the actuator is at least approximately related to the resulting force/torque.

Another approach to torque control is to keep the large gear-ratio motors, but add sensors to directly measure the torque at the joint side of the actuator. This is the approach used by the Kuka iiwa robot that we use in the example throughout this text; the iiwa actuators have strain gauges integrated into the transmission. However there is a trade-off between the stiffness of the transmission and the accuracy of the force/torque measurement Kashiri17 -- the iiwa transmission includes an explicit "Flex Spline" with a stiffness around 5000 Nm/rad Wedler12. Taking this idea to an extreme, Gill Pratt proposed "series-elastic actuators" that have even lower stiffness springs in the transmission, and proposed measuring joint position on both the motor and joint sides of the transmission to estimate the applied torques Pratt95b. For example, the Baxter and Sawyer robots from Rethink used series-elastic actuators; I don't think they ever published the spring stiffness values but similarly-motivated series-elastic actuators from HEBI robotics are closer to 100 Nm/rad. Even for the iiwa actuators, the joint elasticity is significant enough that the low-level controllers go to great length to take it into account explicitly in order to achieve high-performance control of the jointsAlbu-Schaffer07. We will discuss these details when we get to the chapter covering force control.

2.2.3 A proliferation of hardware

The low-cost torque-controlled arms that I mentioned above are just the beginnings in what promises to be a massive proliferation of robotic arms. During the pandemic, I saw a number of people using inexpensive robots like the xArm at home. As demand increases, costs will continue to come down.

Let me just say that, compared to working on legged robots, where for decades we did our research on laboratory prototypes built by graduate students (and occasionally professors!) in the machine shop down the hall, the availability of professionally engineered, high-quality, high-uptime hardware is an absolute treat. This also means that we can test algorithms in one lab and have another lab perhaps at another university testing algorithms on almost identical hardware; this facilitates levels of repeatability and sharing that were impossible before. The fact that the prices are coming down, which will mean many more similar robots in many more labs/environments, is one of the big reasons why I am so optimistic about the next few years in the field.

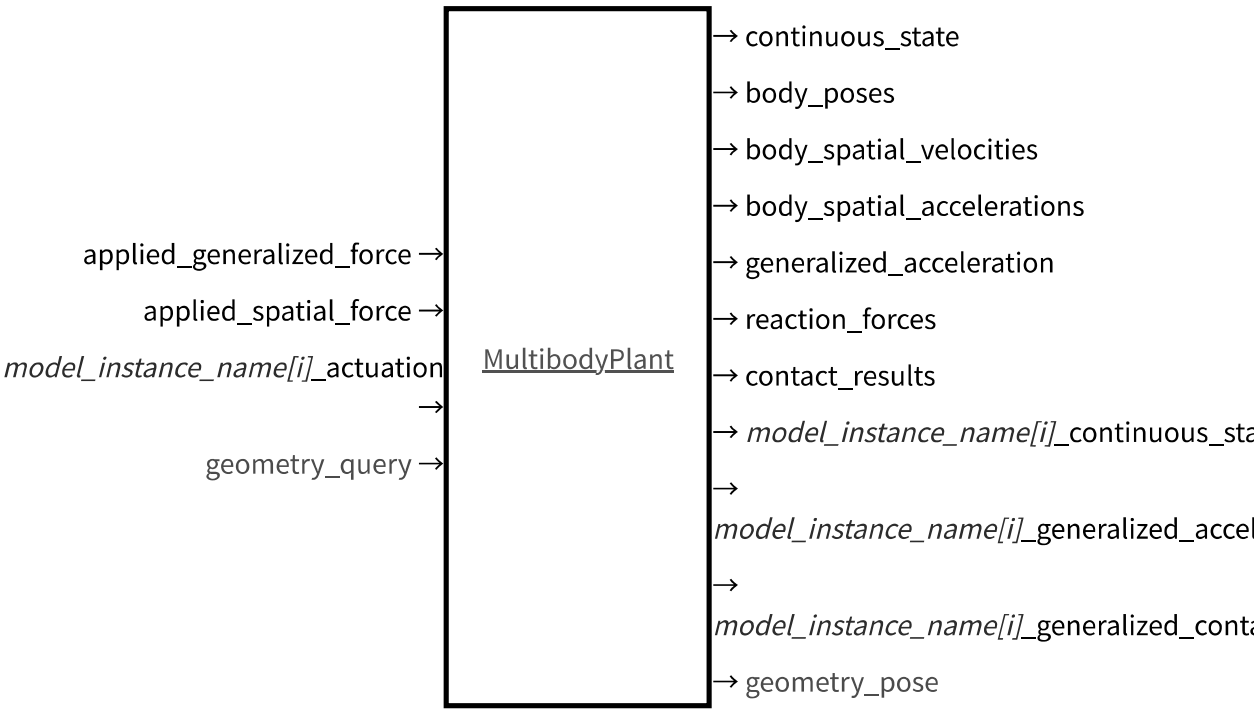
It's a good time to be working on manipulation!

2.2.4 Simulating the Kuka iiwa

It's time to simulate our chosen robotic arm. The first step is to obtain a robot description file (typically URDF or SDF). For convenience, we ship the models for a few robots, including iiwa, with Drake. If you're interested in simulating a different robot, you can find either a URDF or SDF describing most commercial robots somewhere online. But a word of warning: the quality of these models can vary wildly. We've seen surprising errors in even the kinematics (link lengths, geometries, etc), but the dynamics properties (inertia, friction, etc) in particular are often not accurate at all. Sometimes they are not even mathematically consistent (e.g. it is possible to specify an inertial matrix in URDF/SDF which is not physically realizable by any rigid body). Drake will complain if you ask it to load a file with this sort of violation; we would rather alert you early than start generating bogus simulations. There is also increasingly good support for exporting to a robot format directly from CAD software like Solidworks.

Now we have to import this robot description file into our physics engine. In Drake, the physics engine is called `MultibodyPlant`. The term "plant" may seem odd but it is pervasive; it is the word used in the controls literature to represent a physical system to be controlled, which originated in the control of chemical plants. This connection to control theory is very important to me. Not many physics engines in the world go to the lengths that Drake does to make the physics engine compatible with control-theoretic design and analysis.

The `MultibodyPlant` has a class interface with a rich library of methods to work with the kinematics and dynamics of the robot. If you need to compute the location of the center of mass, or a kinematic Jacobian, or any similar queries, then you'll be using this class interface. A `MultibodyPlant` also implements the interface to be used as a `System`, with input and output ports, in Drake's systems framework. In order to simulate, or analyze, the combination of a `MulitbodyPlant` with other systems like our perception, planning, and control systems, we will be assembling block diagrams.



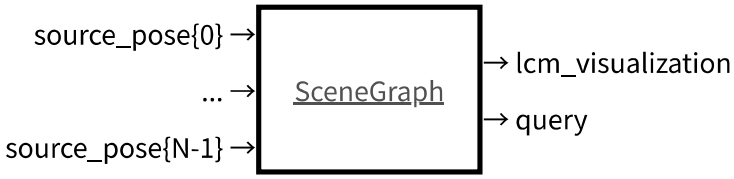
As you might expect for something as complex and general as a physics engine, it has many input and output ports; most of them are optional. I'll illustrate the mechanics of using these in the following example.

Example 2.2 (Simulating the passive iiwa)

It's worth spending a few minutes with this example, which should help you understand not only the physics engine, but some of the basic mechanics of working with simulations in Drake.

Open in Colab

The best way to visualize the results of a physics engine is with a 2D or 3D visualizer. For that, we need to add the system which curates the geometry of a scene; in Drake we call it the `SceneGraph`. Once we have a `SceneGraph`, then there are a number of different visualizers and sensors that we can add to the system to actually render the scene.



Example 2.3 (Visualizing the scene)

This example is far more interesting to watch. Now we have the 3D visualization!

Open in Colab

You might wonder why `MultibodyPlant` doesn't handle the geometry of the scene as well. Well, there are many applications in which we'd like to render complex scenes, and use complex sensors, but supply custom dynamics instead of using the default physics engine. Autonomous driving is a great example; in that case we want to populate a `SceneGraph` with all of the geometry of the vehicles and environment, but we often want to simulate the vehicles with very simple vehicle models that stop well short of adding tire mechanics into our physics engine. We also have a number of examples of this workflow in my Underactuated Robotics course, where we make extensive use of "simple models".

We now have a basic simulation of the iiwa, but already some subtleties emerge. The physics engine needs to be told what torques to apply at the joints. In our example, we apply zero torque, and the robot falls down. In reality, that never happens; in fact there is essentially never a situation where the physical iiwa robot experiences zero torque at the joints, even when the controller is turned off. Like many mature industrial robot arms, iiwa has mechanical brakes at each joint that are engaged whenever the controller is turned off. To simulate the robot with the controller turned off, we would need to tell our physics engine about the torques produced by these brakes.

In fact, even when the controller is turned on, and despite the fact that it is a torque-controlled robot, we can never actually send zero torques to the motors. The iiwa software interface accepts "feed-forward torque" commands, but it will always add these as additional torques to its low-level controller which is compensating for gravity and the motor/transmission mechanics. This often feels frustrating, but probably we don't actually want to get into the details of simulating the drive mechanics.

As a result, the simplest reasonable simulation we can provide of the iiwa must include a simulation of Kuka's low-level controller. We will use the iiwa's "joint impedance control" mode, and will describe the details of that once they become

important for getting the robot to perform better. For now, we can treat it as given, and produce our simplest reasonable iiwa simulation.

Example 2.4 (Adding the iiwa low-level controller)

This example adds the iiwa controller and sets the desired *positions* (no longer the desired torques) to be the current state of the robot. It's a more faithful simulation of the real robot. I'm sorry that it is boring once again!

Open in Colab

As a final note, you might think that simulating the *dynamics* of the robot is overkill, if our only goal is to simulate manipulation tasks where the robot is moving only relatively slowly, and effects of mass, inertia and forces might be less important than just the positions that the robot (and the objects) occupy in space. I would actually agree with you. But it's surprisingly tricky to get a *kinematic* simulation to respect the basic rules of interaction; e.g. to know when the object gets picked up or when it does not (see, for instance Pang18). Currently, in Drake, we mostly use the full physics engine for simulation, but often use simpler models for manipulation planning and control.

2.3 Hands

You might have noticed that the iiwa model does not actually have a hand attached; the robot ships with a mounting plate so that you can attach the "end-effector" of your choice (and some options on access ports so you can connect your end-effector to the computer without wires running down the outside of the robot). So now we have another decision to make: what hand should we use?

Example 2.5 (Robot hands)

We can explore different hand models in Drake using the same sort of interface we used for the arms, though I don't have as many hand models here yet. Let me know if your favorite hand isn't on the list!

Open in Colab

It is interesting that, when it comes to robot end effectors, researchers in manipulation tend to partition themselves into a few distinct camps.

2.3.1 Dexterous hands



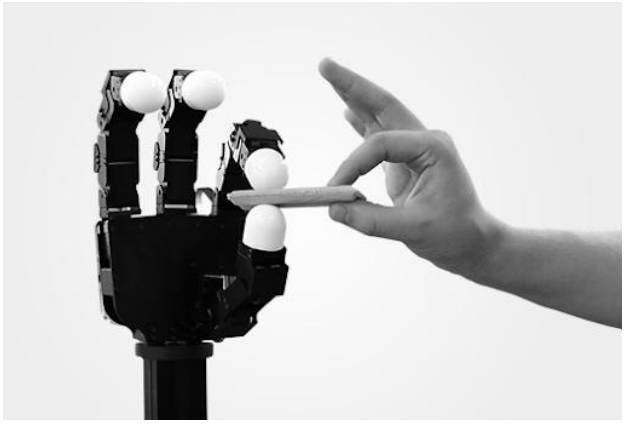


Figure 2.3 - Dexterous hands. Left: the Shadow Dexterous Hand. Right: the Allegro Hand.

Of course, our fascination with the human hand is well placed, and we dream of building robotic hands that are as dexterous and sensor-rich. But the reality is that we aren't there yet. Some people choose to pursue this dream and work with the best dexterous hands on the market, and struggle with the complexity and lack of robustness that ensues. The famous "learning dexterity" project from OpenAI used the Shadow hand for playing with a Rubik's cube, and the work that had to go into the hand in order to support the endurance learning experiments was definitely a part of the story. There is a chance that new manufacturing techniques could really disrupt this space -- videos like this one of FLLEX v2 look amazingKim19 -- and I am very optimistic that we'll have more capable and robust dexterous hands in the not-so-distant future.

2.3.2 Simple grippers



Figure 2.4 - This video of tele-operation with the PR1 from Ken Salisbury's group is now a classic example of doing amazingly useful things with a very simple hand. Check out their website for more videos, including sweeping, fetching a beer, and unloading a dishwasher.



Another camp points out that dexterous hands are not necessary -- I can give you a simple gripper from the toy store and you can still accomplish amazingly useful tasks around the home. The PR1 videos above are a great demonstration of this.

Another important argument in favor of simple hands is the elegance and clarity that comes from reducing the complexity. If thinking clearly about simple grippers helps us understand more deeply *why* we need more dexterous hands (I think it will), then great. For most of these notes, a simple two-fingered gripper will serve our pedagogical goals the best. In particular, I've selected the Schunk WSG 050, which we have used extensively in our research over the last few years. We'll also explore a few different end-effectors in later chapters, when they help to explain the concepts.

To be clear: just because a hand is simple (few degrees of freedom) does not mean that it is low quality. On the contrary, the Schunk WSG is a very high-quality gripper with force control and force measurement at its single degree of freedom that surpasses the fidelity of the Kuka. It would be hard to achieve the same in a dexterous hand with many joints.

2.3.3 Soft/underactuated hands

Finally, the third and newest camp is promoting clever mechanical designs for hands, which are often called "underactuated hands". The basic idea is that, for many tasks, you might not need as many actuators in your hand as you have joints. Many underactuated hands use a cable-drive mechanism to close the fingers, where a single tendon can cause multiple joints in the finger to bend. When designed correctly, these mechanisms can allow the finger to conform passively to the shape of an object being grasped with no change in the actuator command (c.f. Odhner14). Cables are not required for this concept to work; qualitatively similar behavior can be achieved using clever rigid mechanical linkages, as well.



Figure 2.5 - Underactuated hands. Left: the RightHand Robotics Reflex2 is a descendant of the i-HY handOdhner14. Right: the Robotiq 3-fingered gripper.

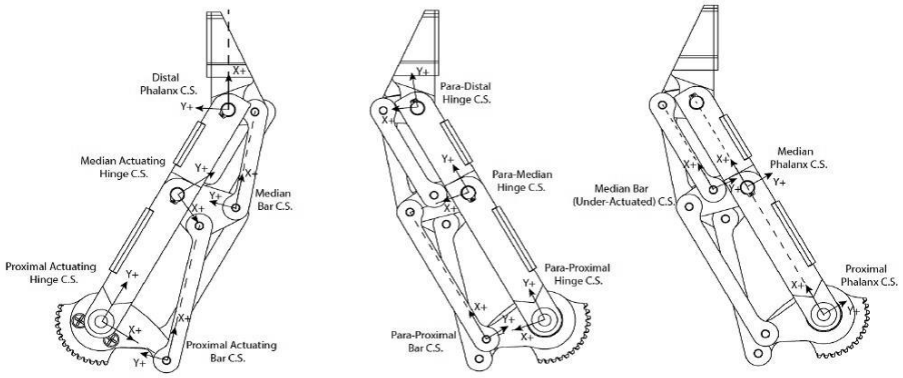


Figure 2.6 - A clever mechanical linkage allows the underactuated Robotiq 3-fingered gripper to comply to an object being grasped.

Taking the idea of underactuation and passive compliance to an extreme, recent years have also seen a number of hands (or at least fingers) that are completely soft. The "soft robotics community" is rapidly changing the state of the art in terms of robot fabrication, with appendages, actuators, sensors, and even power sources that can be completely soft. These technologies promise to improve durability, decrease cost, and potentially be more safe for operating around people.

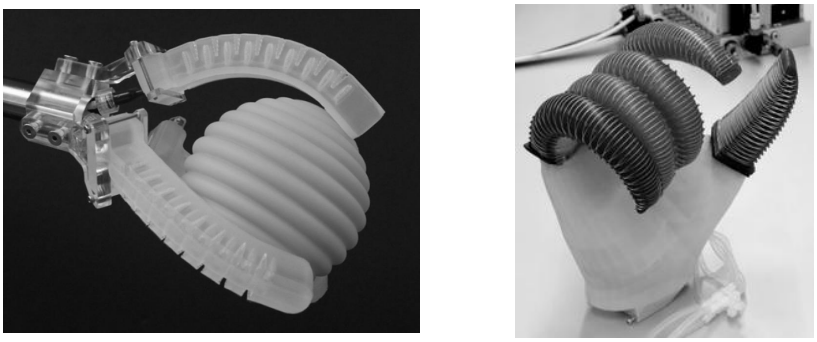


Figure 2.7 - Underactuated hands. Left: A 3D-printed soft hand from Harvard (Image credit: Ryan Truby). Right: The RBO Hand 2 (Image credit: Disney Research Zurich).

Underactuated hands can be excellent examples of mechanical design reducing the burden on the actuators / control system. Often these hands are amazingly good at some range of tasks (most often "enveloping grasps"), but not as general purpose. It would be very hard to use one of these to, for instance, button my shirt. They are, however, becoming more and more dexterous; check out the video below!



2.3.4 Other end effectors

Not all end effectors need to operate like dexterous or simplified human hands. Many industrial applications these days are doing a form of pick and place manipulation using vacuum grippers (also known as suction-cup grippers). Suction cups work extremely well on many but not all objects. Some objects are too soft or porous to be suctioned effectively. Some objects are too fragile or heavy to be lifted from a vacuum at the top of the object, and must be supported from below. Some hands have suction in the palms to achieve an initial pick, but still use more traditional fingers to stabilize a grasp.

There are numerous other clever gripper technologies. One of my favorites is the jamming gripper. These grippers are made of a balloon filled with coffee grounds, or some other granular media; pushing down the balloon around an object allows the granular media to flow around the object, but then applying a vacuum to the balloon causes the granular media to "jam", quickly hardening around the object to make a stable grasp Brown10.



前往平台观看：

Here is another clever design with actuated rollers at the finger tips to help with in-hand reorientation.

Finally, a reasonable argument against dexterous hands is that even humans often do some of their most interesting manipulation not with the hand directly, but through tools. I particularly liked the response that Matt Mason, one of the main advocates for simple grippers throughout the years, gave to a question at the end of one of our robotics seminars: he argued that useful robots in e.g. the kitchen will probably have special purpose tools that can be changed quickly. In applications where the primary job of the dexterous hand is to change tools, we might skip the complexity by mounting a "tool changer" directly to the robot and using tool-changer-compatible tools.

2.3.5 If you haven't seen it...

One time I was attending an event where the registration form asked us "what is your favorite robot of all time, real or fictional". That is a tough question for someone who loves robots! But the answer I gave was a super cool "high-speed multifingered hand" by the Ishikawa group; a project that started turning out amazing results back in 2004! They "overclocked" the hand -- sending more current for short durations than would be reasonable for any longer applications -- and also used high-speed cameras to achieve these results. And they had a Rubik's cube demo, too, in 2017.



前往平台观看：

So good!

2.4 Sensors

I haven't said much yet about sensors. In fact, sensors are going to be a major topic for us when we get to perception with (depth) cameras, and when we think about tactile sensing. But I will defer those topics until we need them.

For now, let us focus on the joint sensors on the robot. Both the iiwa and the Schunk WSG provide joint feedback -- the iiwa driver gives "measured position", "estimated velocity", and "measured torque" at each of its seven joints; remember that joint accelerations are typically considered too noisy to rely on. Similarly the Schunk WSG outputs "measured state" (position + velocity) and "measured force". We can make all of these available as ports in a block diagram.

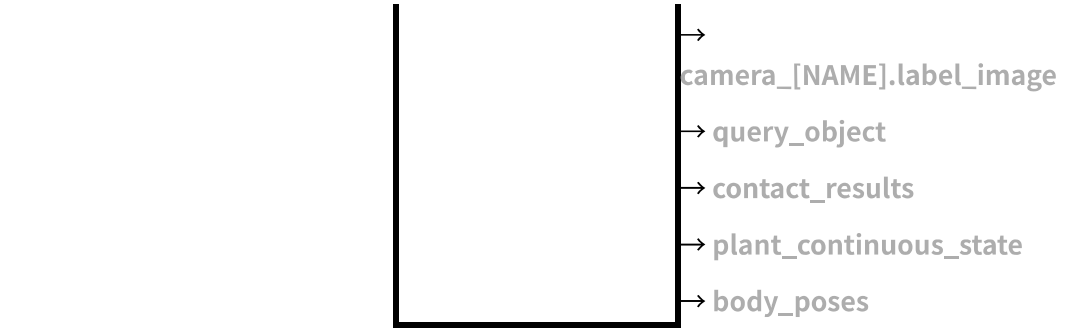
2.5 Putting it all together

If you've worked through the examples, you've seen that a proper simulation of our robot is more than just a physics engine -- it requires assembling physics, actuator and sensor models, and low-level robot controllers into a common framework. In practice, in Drake, that means that we are assembling increasingly sophisticated block diagrams.

2.5.1 HardwareStation

One of the best things about the block-diagram modeling paradigm is the power of abstraction and encapsulation. We can assemble a `Diagram` that contains all of the components necessary to simulate our hardware platform and its environment, which we will refer to affectionately as the "Hardware Station". The method `MakeHardwareStation` takes a YAML description of the scene and the robot hardware. For a yaml file describing the iiwa + WSG and some cameras, the resulting `HardwareStation` system looks like this:





The output ports labeled in in orange on the diagram above are "cheat ports" -- they are available in simulation, but cannot be available when running on the real robot (because they assume a ground-truth knowledge).

Example 2.6 (Hardware station in the teleop demo)

The teleop notebook in the first chapter used the `MakeHardwareStation` interface to set up the simulation. Now you have a better sense for what is going on inside that subsystem! Here is the link if you want to take a look at that example again:

Open in Colab

Example 2.7 (A bimanual manipulation station)

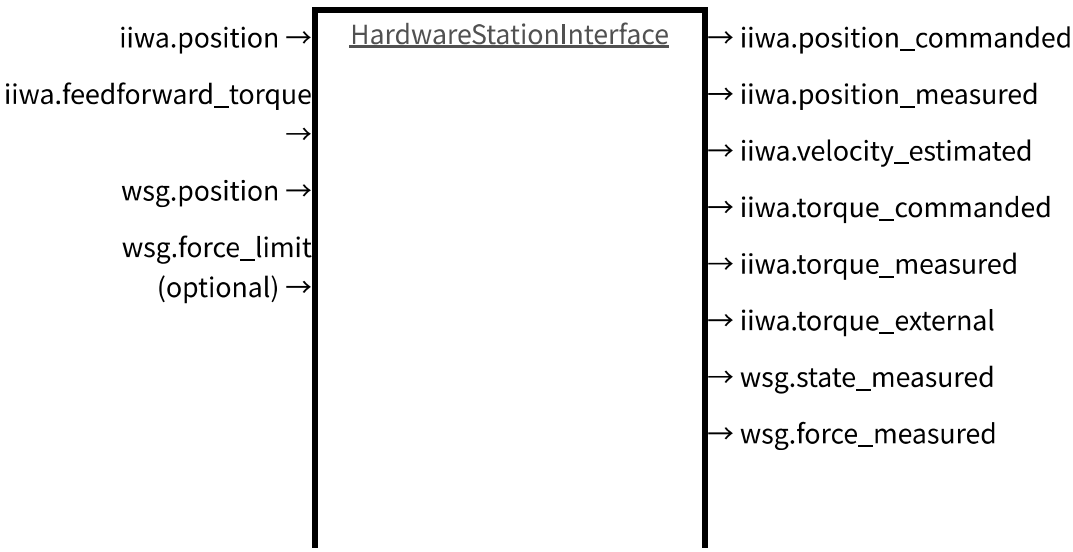
By adding a few more lines to the YAML file, we can use the same `MakeHardwareStation` method to construct a bimanual station:

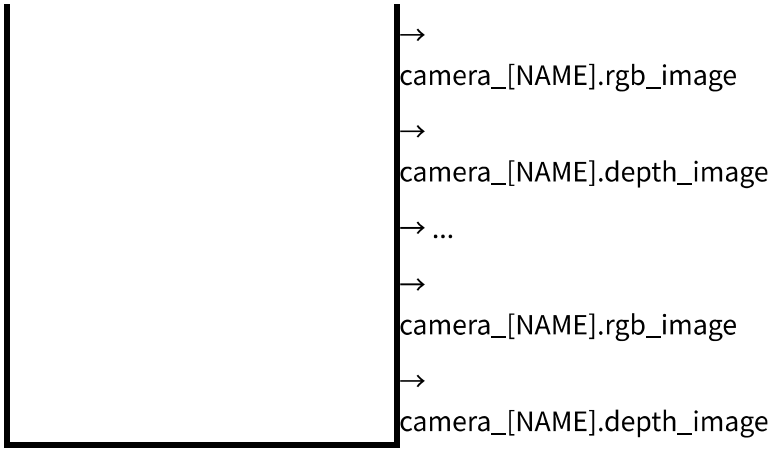
Open in Colab

If there are other robots/drivers that you'd like to simulate, you can make local modifications to the `station.py` file directly in the manipulation repository associated with these notes, or just ask me and I can probably add them quickly.

2.5.2 HardwareStationInterface

As you see in the examples, the `HardwareStation` diagram itself is intended to be used as a `System` in additional diagrams, which can include our perception, planning, and, higher-level control systems. This model also defines the abstraction between the simulation and the real hardware. By simply passing `hardware=True` into the `MakeHardwareStation` method, we instead construct an almost identical system, the `HardwareStationInterface`.





The `HardwareStationInterface` is also a diagram, but rather than being made up of the simulation components like `MultibodyPlant` and `SceneGraph`, it is made up of systems that perform network message passing to interface with the small executables that talk to the individual hardware drivers. If you dig under the covers, you will see that we use LCM for this instead of ROS messages, primarily because LCM is a lighter-weight dependency for our public repository (also because multicast UDP is a better choice that TCP/IP for the driver level interface). But many Drake developers/users use Drake in a ROS/ROS2 ecosystem.

If you do have your own similar robot hardware available, and want to run the hardware interface on your machines, I've started putting together a list of drivers and bill of materials in the appendix.

2.5.3 HardwareStation stand-alone simulation

Using the `HardwareStation` workflow, it is easy to transition from your development in simulation to running on the real robot. One additional tool to support this workflow is the stand-alone `hardware_sim` executable. This python script takes the same YAML file as as input (via the command line), and starts up a simulation in a separate process that acts just like the real robot hardware should... sending and receiving the hardware side of the messages. This can be valuable for testing that all of your logic still works with the message passing layer adding some delay and non-determinism that we cleverly avoid when we use `MakeHardwareStation(..., hardware=False)` in the first stages of our development.

```
python3 drake/examples/hardware_sim/hardware_sim.py
    --scenario_file=station.yaml --scenario_name=Name
```

2.6 More HardwareStation examples

I love the projects that students put together for this class. To help enable those projects (and your future projects, I hope), I will collect a few more examples here of setting up the `HardwareStation` for different hardware configurations. Expect this list to grow over time!

Example 2.8 (The iiwa with an Allegro hand)

Here is a simple example of simulating the iiwa with the Allegro hand attached instead of the Schunk WSG gripper. (Note that there are both left and right versions of the Allegro hand available.)

Open in Colab

2.7 Exercises

Exercise 2.1 Role of Reflected Inertia

For this exercise you will investigate the effect of reflected inertia on the joint-space dynamics of the robot, and how it affects simple position control laws. You will work exclusively in this notebook. You will be asked to complete the following steps:

a. Derive the first-order state-space dynamics $\dot{\bx} = f(\bx, \bu)$ of a simple pendulum with a motor and gearbox.
b. Compare the behavior of the direct-driven simple pendulum and the simple pendulum with a high-ratio gearbox, under the same position control law.

Exercise 2.2 Input and Output Ports on the Manipulation Station

For this exercise you will investigate how a manipulation station is abstracted in Drake's system-level framework. You will work exclusively in this notebook. You will be asked to complete the following steps:

a. Learn how to probe into inputs and output ports of the manipulation station and evaluate their contents.
b. Explore what different ports correspond to by probing their values.

Exercise 2.3 Direct Joint Teleop in Drake

For this exercise you will implement a method for controlling the joints of a robot in Drake. You will work exclusively in this notebook, and should use the example notebook in chapter 1 as a reference. You will be asked to complete the following steps:

a. Replace the teleop interface in the chapter 1 example with different Drake functions that allow for directly controlling the joints of the robot.

Exercise 2.4 PID Control in Drake

For this exercise you will implement a PID controller the joints of a robot in Drake. You will work exclusively in this notebook. You will be asked to complete the following steps:

a. Implementing a PD controller for the joints of the robot.
b. Extending the PD controller to a full PID controller.

References

1. Jemin Hwangbo and Joonho Lee and Alexey Dosovitskiy and Dario Bellicoso and Vassilios Tsounis and Vladlen Koltun and Marco Hutter, "Learning agile and dynamic motor skills for legged robots", *Science Robotics*, vol. 4, no. 26, pp. eaau5872, 2019.

2. Haruhiko Asada and Kamal Youcef-Toumi, "Direct-Drive Robots - Theory and Practice", The MIT Press , 1987.

3. P. M. Wensing and A. Wang and S. Seok and D. Otten and J. Lang and S. Kim, "Proprioceptive Actuator Design in the MIT Cheetah: Impact Mitigation and High-Bandwidth Physical Interaction for Dynamic Legged Robots", *IEEE Transactions on Robotics*, vol. 33, no. 3, pp. 509-522, June, 2017.

4. Navvab Kashiri and JÃ¶rn Malzahn and Nikos Tsagarakis, "On the Sensor Design of Torque Controlled Actuators: A Comparison Study of Strain Gauge and Encoder Based Principles", *IEEE Robotics and Automation Letters*, vol. PP, 02, 2017.

5. A Wedler and M Chalon and K Landzettel and M Görner and E Krämer and R Gruber and A Beyer and HJ Sedlmayr and B Willberg and W Bertleff and others, "DLRs dynamic actuator modules for robotic space applications", *Proceedings of the 41st Aerospace Mechanisms Symposium* , May 16-18, 2012.

6. G. A. Pratt and M. M. Williamson, "Series elastic actuators", *Proceedings 1995 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human Robot Interaction and Cooperative Robots* , vol. 1, pp. 399-406 vol.1, Aug, 1995.

7. Alin Albu-Schaffer and Christian Ott and Gerd Hirzinger, "A unified passivity-based control framework for position, torque and impedance control of flexible joint robots", *The international journal of robotics research*, vol. 26, no. 1, pp. 23--39, 2007.

8. Tao Pang and Russ Tedrake, "A Robust Time-Stepping Scheme for Quasistatic Rigid Multibody Systems", *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* , 2018. [link]

9. Yong-Jae Kim and Junsuk Yoon and Young-Woo Sim, "Fluid Lubricated Dexterous Finger Mechanism for Human-Like Impact Absorbing Capability", *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3971--3978, 2019.

10. Lael U. Odhner and Leif P. Jentoft and Mark R. Claffee and Nicholas Corson and Yaroslav Tenzer and Raymond R. Ma and Martin Buehler and Robert Kohout and Robert D. Howe and Aaron M. Dollar, "A Compliant, Underactuated Hand for Robust Manipulation", *International Journal of Robotics Research (IJRR)*, vol. 33, no. 5, pp. 736-752, 2014.

11. Eric Brown and Nicholas Rodenberg and John Amend and Annan Mozeika and Erik Steltz and Mitchell R Zakin and Hod Lipson and Heinrich M Jaeger, "Universal robotic gripper based on the jamming of granular material", *Proceedings of the National Academy of Sciences*, vol. 107, no. 44, pp. 18809â€“18814, 2010.

Previous Chapter Table of contents Next Chapter

Accessibility

© Russ Tedrake, 2026